

Minecraft Mob Detection

Real-time multi-class object detection

Igor Jakus Jan Pułtorak

16.06.2026

Problem Definition

Given a live gameplay video stream, the model should, **in real time**:

- **Detect** all visible mobs in each frame
- **Classify** each detected mob by type (Creeper, Cow, Pig, Sheep, Chicken)
- **Localize** each detection with a bounding box

Formalization — Input & Classes

1. The input — each frame is a 3D tensor:

$$I \in \mathbb{R}^{H \times W \times 3}$$

where H , W are height/width and 3 are the RGB channels.

2. The target classes — the set of mob types:

$$C = \{c_0, c_1, c_2, \dots, c_K\}$$

with K mob classes; c_0 is the background (no mob).

3. The detections — the model outputs N predictions per frame:

$$\hat{Y} = \{ \hat{d}_1, \hat{d}_2, \dots, \hat{d}_N \}$$

Each detection is a tuple of three components:

$$\hat{d}_i = (\hat{b}_i, \hat{p}_i, \hat{s}_i)$$

$$\hat{b}_i = (\hat{x}_i, \hat{y}_i, \hat{w}_i, \hat{h}_i) \in \mathbb{R}^4 \quad (\text{box})$$

$$\hat{p}_i = (\hat{p}_i^1, \dots, \hat{p}_i^K) \quad (\text{class probs})$$

$$\hat{s}_i \in [0, 1] \quad (\text{confidence})$$

4. The model function — parameterized by weights θ :

$$f_{\theta}(I) = \hat{Y}$$

5. The objective — a multi-task loss compares $\hat{Y}$ to ground truth Y :

$$L = \lambda_{\text{loc}} L_{\text{loc}}(b, \hat{b}) + \lambda_{\text{cls}} L_{\text{cls}}(c, \hat{p})$$

- L_{loc} — bounding-box coordinate error
- L_{cls} — classification error
- $\lambda_{\text{loc}}, \lambda_{\text{cls}}$ — weights balancing the two tasks

Examples



Examples



Why Minecraft?

- **Controlled visual domain** — consistent block-art style, limited palette, predictable lighting
- **No privacy issues** — fully synthetic data
- **Challenging enough** — occlusion, scale variation, many classes, varying biomes & lighting
- **Data availability** — existing datasets
- **Fun & engaging** — motivates thorough experimentation

Source: Minecraft datasets on Roboflow — community-curated, with labeled bounding boxes.

- Pre-annotated images split into train / valid / test
- Inference at 640×640 px (imgsz=640)

Classes (5): pig, chicken, cow, creeper, sheep

Models — fine-tuned with Ultralytics on the Minecraft dataset:

- **YOLOv8n** — lightweight single-stage baseline
- **YOLO26m** — modern middle-ground detector
- **RT-DETR-L** (Large) — transformer-based real-time detector
- **RT-DETR-X** (Extra Large) — larger transformer variant

Compute — University HPC

- 2× **RTX 3090** (multi-GPU, DDP)
- Scheduled via **Slurm** (sbatch)

Tooling & Augmentation

- PyTorch + Ultralytics
- Tracking: Weights & Biases
- Mosaic, flips, brightness/contrast

The trained model runs on **live Minecraft gameplay**, annotating each frame on the fly:

```
python scripts/detect_video.py gameplay.mp4 \  
    --model weights/rtdetr-x.pt \  
    --conf 0.4 --imgsz 640
```

- Frame-by-frame streaming (memory-efficient generator)
- Bounding boxes + class + confidence drawn per frame

Results

Model	mAP@0.5	mAP@0.5:0.95	P	R	FPS	Train time
YOLOv8n	89.5%	67%	93.9%	81.4%	285.6	10.3 min
YOLO26m	92%	68.3%	86.8%	90.6%	92.1	22.1 min
RT-DETR-L	92.2%	69.3%	88.9%	89.9%	80	47.9 min
RT-DETR-X	84.8%	62%	86.8%	80.9%	83.2	87.4 min

All runs: 100 epochs, 2× RTX 3090, 640×640 px. P = precision, R = recall. FPS = PyTorch inference.

Analysis & Discussion

- **RT-DETR-L** — best accuracy (92.2% mAP@0.5); **YOLO26m** nearly matches it and trains 2× faster — better tradeoff for real-time gameplay
- **YOLOv8n** — fastest (286 FPS) but lowest recall (81%) — conservative, misses more mobs
- **RT-DETR-X** underperformed despite size — likely smaller batch (8 vs 16) and GPU memory limits
- mAP@0.5:0.95 (62–69%) lags mAP@0.5 — **tight bounding boxes** are harder than finding mobs

Limitations: 5 classes only; FPS is a PyTorch benchmark, not full gameplay pipeline; batch sizes varied across models

Conclusions

- Built an **end-to-end real-time** Minecraft mob detector (5 classes)
- **RT-DETR-L** was our best model (mAP@0.5 $\approx$ 92%); YOLO26m close behind and much faster to train
- Runs live on **gameplay video**

Thank you!
Questions?